

Configuring GitHub Deployment Secrets — Portfolio 5

Step-by-step instructions for adding the secrets required by `.github/workflows/deploy.yml` and `.github/workflows/release.yml`. No secret values are stored in this repo — this is a setup guide only. Must be done by someone with **admin access** to `Trust-Internship/trust-digital-communication` on GitHub.

1. Secrets needed, per environment

Environment	Secret name	Purpose
development	DEV_HOST	Dev server hostname/IP
	DEV_USER	SSH login user on the Dev server
	DEV_SSH_KEY	Private SSH key for DEV_USER (SCP + deploy steps)
	DEV_PORT	SSH port (usually 22)
	DEV_CONNECTION_STRING	Postgres connection string written into <code>appsettings.Production.json</code> on Dev
	DEV_JWT_SECRET	JWT signing secret written into <code>appsettings.Production.json</code> on Dev
uat	UAT_HOST	UAT server hostname/IP
	UAT_USER	SSH login user on the UAT server
	UAT_SSH_KEY	Private SSH key for UAT_USER
	UAT_PORT	SSH port
	UAT_CONNECTION_STRING	Postgres connection string for UAT
	UAT_JWT_SECRET	JWT signing secret for UAT
production	PROD_HOST	Production server hostname/IP
	PROD_USER	SSH login user on the Production server
	PROD_SSH_KEY	Private SSH key for PROD_USER
	PROD_PORT	SSH port
	PROD_CONNECTION_STRING	Postgres connection string for Production
	PROD_JWT_SECRET	JWT signing secret for Production

`development` is used by both `deploy.yml` (push to `main`) and `release.yml` (version tags). `uat` and `production` are only used by `release.yml`.

2. Create the GitHub Environments (if not already present)

1. GitHub → repo → **Settings** → **Environments** → **New environment**.
2. Create three, named exactly: **development**, **uat**, **production** (must match the **environment:** keys in the workflow files exactly, case-sensitive).
3. For **production** only: under **Deployment protection rules**, enable **Required reviewers** and add whoever must approve production deploys. This is already expected by **release.yml** (see the comment on the **deploy-prod** job).

3. Generate an SSH deploy keypair (repeat per environment)

Run locally (do **not** reuse your personal SSH key):

```
ssh-keygen -t ed25519 -C "github-actions-deploy-dev" -f dev_deploy_key -N ""
```

This creates two files:

- **dev_deploy_key** — the private key (goes into GitHub as **DEV_SSH_KEY**)
- **dev_deploy_key.pub** — the public key (goes onto the target server)

Repeat with different filenames/comments for **uat** and **production** (**uat_deploy_key**, **prod_deploy_key**) — **use a separate keypair per environment** so revoking one doesn't affect the others.

4. Authorize the public key on the target server

For each environment's server, log in as the account the workflow will use (**DEV_USER** / **UAT_USER** / **PROD_USER**) and append the matching **.pub** file:

```
ssh <that-user>@<host> "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >>
~/.ssh/authorized_keys" < dev_deploy_key.pub
ssh <that-user>@<host> "chmod 600 ~/.ssh/authorized_keys"
```

Confirm you can SSH in using the new private key before moving on:

```
ssh -i dev_deploy_key <that-user>@<host>
```

5. Add each secret in GitHub

For each environment:

1. GitHub → repo → **Settings** → **Environments** → click the environment name (e.g. **development**) → **Environment secrets** → **Add secret**.
2. Add each row from the table in §1 as its own secret:
 - **Name:** exact name from the table (e.g. **DEV_SSH_KEY**)
 - **Value:**
 - For ***_HOST**, ***_USER**, ***_PORT**: plain text value.

- For `*_SSH_KEY`: paste the **entire contents** of the private key file (e.g. `dev_deploy_key`), including the `-----BEGIN OPENSSH PRIVATE KEY-----` and `-----END OPENSSH PRIVATE KEY-----` lines, exactly as generated — no extra whitespace or line removal.
 - For `*_CONNECTION_STRING`: a full Npgsql connection string, e.g. `Host=<db-host>;Port=5432;Database=trust_dc;Username=<db-user>;Password=<db-password>`
 - For `*_JWT_SECRET`: a random string at least 32 characters long (must match whatever the token-issuing side, Portfolio 3, signs with — confirm with that team before setting this).
3. Repeat for all three environments — secrets are **not** shared across environments even if the values happen to be the same; each environment's secrets must be added separately.

These must be **Environment secrets**, not repository-level (`Settings` → `Secrets and variables` → `Actions`) secrets — the workflow jobs pin `environment: development / uat / production`, and environment secrets only resolve when the job runs under that matching environment.

6. Delete local key files once uploaded

After pasting each private key into GitHub, delete the local `*_deploy_key` files. Don't commit them, don't leave them on a shared machine, don't email them.

7. Verify

1. Trigger the workflow: push to `main` (for `deploy.yml`) or push a version tag like `v1.0.0` (for `release.yml`).
2. Watch the run in GitHub → **Actions**. The `deploy-dev` (and `deploy-uat/deploy-prod` for releases) job's SCP and SSH steps should complete without the `can't connect without a private SSH key or password` error.
3. If a step still fails, re-check: secret name spelling, correct environment scope, and that the public key really is in `authorized_keys` for the exact user named in `*_USER`.

8. Related

See `docs/DEPLOYMENT_SSH_SECRET_ISSUE.md` for the specific error this setup resolves, and `docs/HANDOVER.md` §5 for the separate, unrelated `ConnectionStrings__DefaultConnection` CI (not deployment) fix.